OrchestraBench: Evaluating Multi-Agent Orchestration Failure Modes, Recovery, and Decomposition Quality

Anonymous Author(s)

ABSTRACT

Multi-agent orchestration frameworks (AutoGen, LangGraph, CrewAI, Anthropic Agents SDK) are moving from demos to production, yet they can only be compared on *task accuracy* — not on *reliability*. Existing agent benchmarks measure whether a pipeline succeeds, but cannot diagnose *why* it failed, *where* a cascade began, or *which* routing decision caused the breakdown. We present **OrchestraBench**, a benchmark that evaluates how multi-agent systems **fail, recover, and decompose** as first-class, comparable metrics. OrchestraBench contributes (i) a controlled, seed-reproducible **failure-injection harness** over templated enterprise workflows, (ii) **cascade-radius** and per-failure-mode recovery as primary metrics, and (iii) a **routing-policy comparison** with bootstrap confidence intervals and paired tests where applicable. Our first experiment isolates the routing-mechanism question: on a 26-case gold-labelled diagnostic, a keyword/flag heuristic — representative of production rule-based routers — scores **0% on adversarial cases** (misleading or missing surface flags), while a model-driven router that reasons over intent scores **100%**, matching the oracle on this diagnostic. This indicates that the heuristic's blind spot is a property of the routing *mechanism*, not the diagnostic's inherent difficulty. Two further experiments, run with a real Claude agent over a **verifiable arithmetic dependency chain** as a controlled mechanism probe, isolate a reliability axis existing benchmarks leave undermeasured. Across five MAST failure modes, failure handling splits into **three tiers** — a tool fault is fully recovered (**1.0**), ambiguous delegation is *partially* recovered (**0.30**), and three latent/semantic modes never recover (**0.0**). Two behaviors make this a property of the model rather than of the construct: the tier **ordering survives reframing** the identical computation as a loan-approval workflow while absolute rates shift with context (tool 1.0 → 0.7, ambiguous 0.30 → 0.40), and **retry does not repair the latent modes** — it reproduces the fault and only lengthens time-to-detection, so detection/attribution, not blind retry, is the necessary containment mechanism. As a corroborating structural signature, cascade radius grows with pipeline depth (mean 0.9 to 4.7 across depths 3–7). A policy-conditioned probe asks whether trusted-state repair can contain these cascades; an ablation shows its apparent gain is largely the trusted-state signal rather than autonomous detection, so we report it as a trusted-state probe, not a deployable routing result. We frame these as controlled-chain mechanism probes, not domain-workload claims (§7).

CCS CONCEPTS

• **Computing methodologies** → **Multi-agent systems**; • **General and reference** → **Evaluation**; • **Software and its engineering** → *Software reliability*.

KEYWORDS

LLM orchestration, multi-agent systems, benchmarking, failure recovery, cascade propagation

1 INTRODUCTION

Multi-agent systems increasingly orchestrate several specialized agents — routers, retrievers, tool-callers, planners — behind a single task. When such a pipeline produces a wrong or low-quality answer in production, the failure is often *silent*: a misrouted task still yields a plausible-looking response. Current benchmarks (AgentBench [7], OdysseyBench [9], SWE-Bench [5]) report final task success and so cannot tell teams **which** orchestration decision failed, **how far** an error propagated, or **whether** intelligent routing was worth its added complexity. This blocks the most basic production decision: choosing and hardening an orchestration framework on *reliability* rather than ergonomics.

Contributions.

(1) **OrchestraBench**, a benchmark that treats orchestration *failure handling* as the unit of evaluation: controlled failure injection, per-failure-mode recovery, and cascade propagation.
(2) A **seed-reproducible** workflow + failure-injection harness (scenarios regenerable from seeds; aggregates recomputable from committed artifacts) with templated enterprise workflow families.
(3) A **routing-policy comparison** (Fixed / Heuristic / LLM / Oracle) with bootstrap confidence intervals and paired significance testing where applicable, isolating the routing *mechanism* effect on adversarial cases (Exp 1); the graded "when does routing pay off" comparison on larger workflow suites is future work.

What we find. On a controlled arithmetic chain, orchestration failure handling splits into three tiers by fault type (tool faults recovered, ambiguous delegation partially, three latent/semantic modes never). The result we read as central is not the split itself but its *robustness*: the tier ordering survives reframing the identical computation as a business loan-approval workflow while absolute rates move with context, and *retry does not repair the latent modes* — evidence that failure handling is model behavior under context, not an artifact of the construct. Cascade radius growing with pipeline depth (0.9 to 4.7 across depths 3–7) corroborates this on the same chain but is partly structural by construction, so we report it as a supporting signature rather than a headline (§7).

Research questions.

- **RQ1 (routing value)**: When does intelligent routing (heuristic or LLM) actually beat a zero-decision baseline? *(Partially: Exp 1 isolates the mechanism gap; a graded answer needs the larger suite.)*

- **RQ2 (mechanism)**: Is routing reliability a property of task *difficulty*, or of the routing *mechanism* (keyword/flag matching vs. reasoning over intent)? *(Answered by Exp 1.)*
- **RQ3 (attribution)**: Can a pipeline failure be attributed to a specific failure mode *and* stage, reproducibly across runs?
- **RQ4 (cascade)**: How far does a single seeded error propagate downstream, and which routing/recovery strategies contain it?

2 RELATED WORK

OrchestraBench sits among recent work that *observes*, *attributes*, or *proposes* fixes for multi-agent failure; we differ by making controlled injection, cascade radius, and routing-policy mechanism the unit of analysis (Table 1).

Motivating data point. Beyond the Strongest [8] reports that on GPQA-Diamond at least one agent was correct in **95.5%** of cases, yet orchestration reached only **87.4%** — i.e. orchestration *discards* ~8 points of individually-recoverable correctness. OrchestraBench is built to attribute exactly this loss.

The gap. Existing work either *observes* failures (MAST [2]), *attributes* them post-hoc (TraceElephant [3], [12]), or *proposes* better orchestration (AdaptOrch [11]). The one work that also *injects* (MAS-FIRE [4]) scores reliability but does not make cascade radius or routing-policy mechanism its unit of analysis. OrchestraBench's contribution is a **controlled, reproducible injection harness that measures recovery and cascade containment as first-class metrics, compared across routing policies.**

2.1 Positioning Relative to Closest Prior Work

An honest audit (the space moved fast in early 2026) surfaces one direct overlap and our response:

- **MAS-FIRE [4] already performs controlled fault injection + reliability evaluation**, overlapping our Exp 2/3 substantially. Our differentiation is *not* "we inject failures" (they do too); it is (a) the **behavioral characterization** of failure handling — the sharp tool-vs-latent recovery split, its persistence under domain reframing, and retry's inability to repair latent faults; (b) **cascade radius as a stages-traversed metric across pipeline depths**, which MAS-FIRE does not quantify (we report it honestly as partly structural on our verifiable chain); and (c) the **routing-mechanism result** of Exp 1. In our reading, MAS-FIRE does not quantify cascade depth as a stages-traversed metric and does not vary routing policy; its axes are topology (MetaGPT / Table-Critic / CAMEL), fault-type (15), and model. Our per-routing-policy containment is a probe rather than a headline (the §5.5 ablation shows the LLM policy's gain is largely the trusted-state signal). OrchestraBench therefore complements MAS-FIRE by adding a behavioral, depth-resolved view of the same failure taxonomy.
- **Attribution benchmarks [3, 12] are observational/post-hoc** — a different instrument from seeded, reproducible injection + recovery.
- **Internal overlap: IntentBench [1]** evaluates whether the agent's *understanding of intent* is correct (intent-spec vs. syntactic tool-call correctness); OrchestraBench evaluates which *orchestration action* to take given a correct understanding and how those decisions cascade. We measure the **decision policy and its failure propagation**, not intent correctness — complementary ends of the pipeline, no measurement overlap.

3 THE ORCHESTRABENCH BENCHMARK

OrchestraBench is released as a reusable artifact — a seed-controlled workflow generator, the `FailureInjector`, the routing-policy suite, and the analysis scripts — paired with the initial mechanism-probe results reported here; the broad multi-domain evaluation it is designed to host is future work.

Workflow suites. Synthetically generated from templated task graphs (stages, inter-task dependencies, per-task `complexity_score`), author-screened for realism. Three enterprise families: finance approval (4 stages), HR onboarding (5 stages), DevOps deployment (5 stages).

Routing decision space. Each task is routed to one of four actions: `DIRECT_TOOL`, `CODE_EXECUTION`, `DECOMPOSE`, `REASON_ONLY`.

Gold labels. For the failure-recovery, cascade, and decomposition experiments (Exp 2–4), ground truth is *objective*: an exact-match check against the verifiable arithmetic result, requiring no subjective annotation. The Exp 1 routing diagnostic (26 cases) is author-labelled from unambiguous task intent; a two-annotator protocol with reported inter-annotator agreement (κ) remains future work for the larger workflow-suite evaluation (§7).

Failure injection. Failure scenarios are seeded **deterministically** on top of the suites via the `FailureInjector`, so each scenario is reproducible from a seed.

Metrics. Primary reported metrics are routing accuracy, per-failure-mode recovery rate, **cascade radius**, time-to-detection, recovery completeness, and decomposition delegation fidelity. The harness also records routing macro-F1, per-class routing metrics, success rate, latency, cost, and orchestration efficiency for larger suite-level comparisons.

4 EXPERIMENTAL SETUP

Reported confidence intervals use 95% bootstrap CIs. For the paired comparisons (Exp 4; the policy ablation) we report an *exact* two-sided sign-flip permutation p ($\alpha = 0.05$): with small, often-separated paired samples the percentile-bootstrap p is anti-conservative (it collapses to 0 when every paired difference shares a sign), so the permutation test is the honest instrument. Per-mode aggregates are n=30 (Exp 2) and n=120/depth (Exp 3), pooling the baseline policies. The fully-deterministic cells (e.g. context pollution at final-task success 0.0) remain point masses regardless of n — a corrupted latent state always fails the final task — so their CIs are zero-width by construction; the stochastic cells (ambiguous delegation; the domain framings) carry informative CIs (§7).

Table 1: OrchestraBench versus closely related work.

Prior work	What it does	How OrchestraBench differs
MAST [2] (14 failure modes, 1,600+ traces, κ=0.88)	Empirically-grounded *taxonomy* of observed MAS failures	We *inject* that taxonomy under seed-controlled conditions and measure recovery + cascade *per mode* — MAST observes, we intervene
MAS-FIRE [4]	Fault injection + reliability evaluation; intra-/inter-agent fault taxonomy	**Closest prior work** (§2.1). We differ on emphasis: cascade *radius across pipeline depths + routing-policy comparison* as the unit of analysis, not reliability scoring alone
TraceElephant [3] (220 annotated traces)	Benchmark for post-hoc failure attribution in MAS	Attribution is observational; we add controlled seeding + cascade radius as *primary* metrics
Agents Failure Attribution [12] (Who&When; 127 MAS)	Post-hoc attribution to the responsible agent/step	Same: attribution is a means here, not the product
Orchestration-pattern benchmark [6] (10k SEC filings)	Compares sequential / parallel / hierarchical / reflexive architectures on a cost-accuracy Pareto	Architecture comparison on accuracy/cost; we evaluate *failure handling* and routing *mechanism* — complementary
AdaptOrch [11]	Task-adaptive orchestration *framework* (+12–23%)	A method; OrchestraBench is the benchmark that would evaluate it
From Spark to Fire [10]	Errors cascade exponentially in pipelines	We operationalize this into a measurable cascade-radius benchmark

Table 2: Routing policies under comparison.

Policy	Type	What it tests
Fixed	Baseline	Zero-intelligence lower bound (always one route)
Heuristic	Baseline	Rule/keyword routing on complexity + flags
LLM-as-Router	Test	Model decides routing per task by reasoning over description + metadata
Retry(heuristic)	Test	Adds automatic retry — does retry alone contain cascades?
Oracle	Ceiling	Routes to the gold label (upper bound)

Table 3: Experiment 1: routing accuracy by case type.

Policy	Overall	Aligned	Adversarial
Fixed (no signal)	23%	25%	20%
Heuristic (keyword/flags)	62%	**100%**	**0%**
TF-IDF (reads description)	92%	88%	**100%**
LLM-as-Router (Sonnet 4.6)	**100%**	100%	**100%**
Oracle (ceiling)	100%	100%	100%

5 RESULTS

5.1 Experiment 1 — Routing Policy Comparison (measured)

We isolate the routing-mechanism question on a focused diagnostic of **26 expert-labelled gold cases** covering all four routing decisions: 16 *aligned* (surface flags match intent) and 10 *adversarial* (flags missing or misleading, but the description's intent is unambiguous). The model-driven router is Claude Sonnet 4.6 via forced structured (tool-use) output; results are stable across 3 independent passes (78/78), and the prompt never sees the gold label.

Finding. The heuristic is perfect on aligned cases but collapses to **0%** on adversarial ones; the model-driven router recovers **all** of them (0% → 100%; Wilson 95% CI on the 10 adversarial cases $[0.00, 0.28]$ vs. $[0.72, 1.00]$), matching the oracle on this diagnostic. The contrast is not merely keyword-versus-LLM: a **mid-strength TF-IDF router** that simply reads the task *description* against fixed per-route prototypes — no model, no learned weights — already recovers every adversarial case (100%), while giving back a few aligned cases where the surface flags were in fact the cleaner signal (88%). The takeaway is not a smooth "more reading is better" gradient: surface-flag matching actually scores *below* the no-signal baseline on adversarial cases (0% vs. 20%), since misleading flags are worse than none. The sharp line is *what* the router reads — any router that reads the task *description* clears the adversarial set (TF-IDF and LLM both 100%), while flag-matching alone collapses to 0% (Fig. 1). Because a shallow text router already clears the adversarial set, the heuristic's collapse is a property of the routing **mechanism** — matching surface flags that, by construction, decouple from intent — not of inherent task difficulty. The adversarial set is deliberately a worst case for surface-flag matching; we read Exp 1 as isolating *when* keyword routing fails (flags decoupled from intent), not as a general difficulty-graded benchmark. Larger workflow-suite evaluation is the intended setting for a graded comparison once the diagnostic no longer saturates.

5.2 Experiment 2 — Failure Injection and Recovery (real Claude measured run; core)

Design. Five MAST failure modes — *ambiguous delegation*, *tool-invocation error*, *context pollution*, *conflicting sub-agent outputs*, *premature action* — injected at the **prompt level** into a verifiable arithmetic dependency chain executed by a **real Claude agent** (Sonnet 4.6), under fixed, heuristic, and retry policies. Real recovery

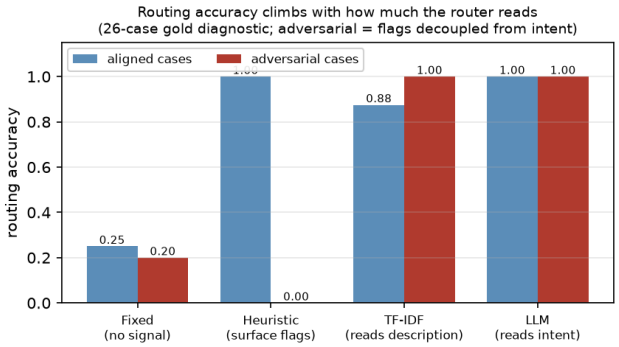


Figure 1: Routing accuracy on the 26-case diagnostic climbs with how much of the task the router reads. The flag heuristic's aligned/adversarial cliff (100%/0%) is flattened by a description-reading TF-IDF router (88%/100%) with no model, isolating the heuristic's blind spot as a mechanism property.

Table 4: Experiment 2: per-mode final-task success and cascade radius, arithmetic chain vs. loan-approval domain (real Claude, N=150 each, n=30/mode; 95% bootstrap CI).

	Arithmetic		Domain	
Failure mode	**Success**	**Cascade**	**Success**	**Cascade**
tool_invocation_error	1.00	0.00	0.70	0.60
ambiguous_delegation	0.30	1.40	0.40	1.20
context_pollution	0.00	2.00	0.00	2.00
conflicting_outputs	0.00	2.00	0.00	2.00
premature_action	0.00	2.00	0.00	2.00

/ cascade are measured by exact-match against ground truth; the measured-run module is included in the repository.

Real measured findings (Sonnet 4.6, N=150, n=30/mode). Failure handling splits into **three tiers**. (i) The tool-invocation mode is **fully recovered** (recovery 1.0, cascade radius 0) — the agent recomputes by hand when the tool fails. (ii) **Ambiguous delegation is partially recovered** (final-task success **0.30 [0.13, 0.47]**, cascade radius 1.40) — with n=30 the agent infers the intended operation about a third of the time, a genuinely *stochastic* cell rather than a point mass. (iii) The remaining three latent/semantic modes (context pollution, conflicting outputs, premature action) **never recover** (final-task success **0.0**) and cascade to every downstream stage. Critically, the retry policy **does not recover the latent modes**: retrying while the failure is still present reproduces the error and only **lengthens time-to-detection** — retry repairs retryable tool faults but not failures needing attribution, state repair, or semantic validation (RQ3/RQ4). The stochastic cells now carry informative CIs (ambiguous 0.30 [0.13, 0.47]); the fully-deterministic 0.0 / 1.0 cells remain point masses by construction (§7).

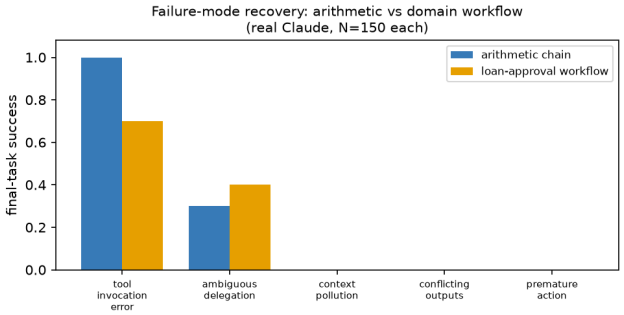


Figure 2: Experiment 2 — failure-mode final-task success, arithmetic chain vs. loan-approval workflow; the failure-mode *ordering* is robust across framings while absolute rates shift (real Claude, N=150 each).

Table 5: Experiment 3: mean cascade radius by pipeline depth (real Claude, N=750; latent modes pooled, n=120/depth; 95% bootstrap CI). Latent cascade radius grows monotonically with depth: the three catastrophic modes track depth − 2 exactly, while ambiguous delegation runs lower as it partially recovers.

Depth	Latent cascade radius	Tool cascade radius
3	0.93 [0.88, 0.97]	0.00
4	1.85 [1.75, 1.93]	0.00
5	2.80 [2.65, 2.93]	0.00
6	3.63 [3.43, 3.83]	0.00
7	4.67 [4.42, 4.88]	0.00

Domain-grounded validation (loan-approval workflow, N=150). To test whether these signatures are an artifact of the abstract arithmetic chain, we re-ran the *identical* failure injection on a domain-grounded variant: the same verifiable computation reframed as a multi-role loan-approval pipeline (Intake Officer → Risk Analyst → Compliance Officer → Approval Manager), each stage prompted in business terms. **The core ordering holds**: context pollution, conflicting outputs, and premature action still fail completely, while tool faults remain the most recoverable (Figure 2). Crucially, the **absolute rates shift with framing**: ambiguous delegation rises from **0.30** to **0.40** (the business-role context helps the agent infer the intended operation) while tool-invocation recovery falls from **1.00** to **0.70**. This framing-sensitivity is evidence of model behavior under context, not a purely structural tautology, and directly addresses the construct-validity concern (§7).

5.3 Experiment 3 — Cascade Propagation Depth (real Claude measured run, N=750; core)

Design. Inject a single seeded error at stage 1 of variable-depth pipelines (depths 3 through 7) and measure how far it propagates. **Cascade radius** (downstream stages corrupted) is a differentiator vs. MAS-FIRE, which has no stages-traversed metric.

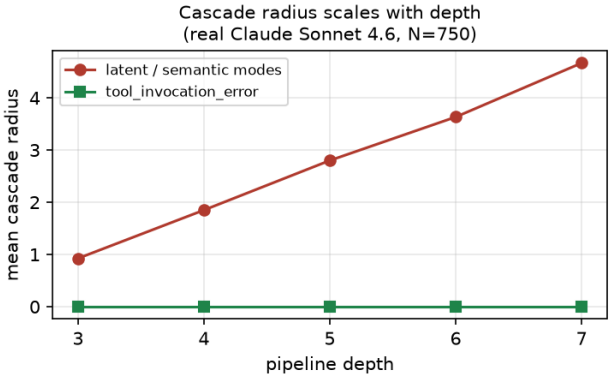


Figure 3: Experiment 3 — mean cascade radius vs. pipeline depth; latent/semantic modes scale 0.93 → 4.67 across depths 3–7 while tool faults stay at 0 (real Claude, N=750).

Table 6: Experiment 4: decomposition quality, decompose vs. monolithic (real Claude, N=40; 95% bootstrap CI; exact permutation p over the 20 shared tasks). Zero-width CIs are degenerate by construction (§4).

Policy	Deleg. fidelity	Granularity err.	Final correct
decompose	1.00 [1.00, 1.00]	0.00 [0.00, 0.00]	1.00
monolithic	0.37 [0.33, 0.42]	2.00 [2.00, 2.00]	1.00
paired Δ	fidelity +0.63 [0.58, 0.67], p=1.9×10^{-6}; granularity −2.0, p=1.9×10^{-6}		

Real measured findings (Sonnet 4.6, N=750, n=120/depth). Pooled over the latent modes, **mean cascade radius grows monotonically with depth: 0.93 → 1.85 → 2.80 → 3.63 → 4.67 across depths 3 / 4 / 5 / 6 / 7** (Table 5, Figure 3), an almost-linear ∼0.9-per-stage trend now resolved at five depths with non-degenerate CIs. The structure is sharp per mode: the three catastrophic modes (context pollution, conflicting outputs, premature action) corrupt *every* downstream stage (cascade = depth − 2 exactly), while ambiguous delegation runs consistently lower (0.7 / 1.4 / 2.2 / 2.5 / 3.7) because it partially recovers, and tool-invocation errors stay at 0 at every depth (the agent recomputes by hand). **This quantifies a stages-traversed signature MAS-FIRE leaves unmeasured**; on the verifiable chain the deterministic modes' growth is partly structural (§7), so we read the depth-scaling as corroborating rather than as the paper's headline.

5.4 Experiment 4 — Decomposition Quality (real Claude measured run, N=40; secondary)

For composite tasks $(a\,op\,b)\,op\,(c\,op\,d)$ with a canonical 3-step gold decomposition, we score the produced decomposition against gold — **delegation fidelity** (gold sub-results recovered), **granularity error**, **wasted sub-tasks** — comparing a `monolithic` (single-step) vs. a `decompose` (planner) policy.

Table 7: Policy-conditioned containment on latent failures (real Claude, N=75; 95% bootstrap CI).

Policy	Latent recovery	Cascade radius
fixed / heuristic / retry	0.08 [0.00, 0.25]	1.83 [1.50, 2.00]
LLM-as-router	**0.83 [0.58, 1.00]**	**0.33 [0.00, 0.83]**
Oracle (ceiling)	1.00 [1.00, 1.00]	0.00 [0.00, 0.00]

Real measured findings (Sonnet 4.6, N=40). The `decompose` policy produces a faithful three-step decomposition (delegation fidelity **1.00**, granularity error **0**), while `monolithic` reaches the correct final answer but exposes no delegable sub-structure (fidelity **0.37 [0.33, 0.42]**, granularity error **2**). This fidelity gap is large and statistically significant under an exact sign-flip permutation test over the 20 shared tasks: +0.63**, 95% CI [0.58, 0.67],** p=1.9×10^{-6} (Table 6). Notably, `final_correct` is **1.0 for both** — the arithmetic is easy enough that both get the answer; the discriminator is the **decomposition structure**, not final correctness. For multi-agent orchestration this is exactly the point: a monolithic agent can be *right* yet produce nothing a planner can delegate, audit, or recover from.

5.5 Policy-conditioned containment probe (real Claude, N=75 + N=225)

Experiments 2–3 pool over baseline policies; we add **LLM-as-router** and **Oracle** policies to ask whether the routing policy itself can govern containment. **The LLM-policy semantics are an explicit modelling assumption**: at the injected stage the router is given the trusted upstream value and licensed to detect and correct the anomaly. We therefore treat the LLM row as a strong self-correction probe rather than a clean estimate of deployable routing without trusted state.

Policy probe by depth (N=225). Under this stronger trusted-state semantics, the policy effect *amplifies with pipeline depth*: baseline cascade radius grows with depth (0.9 / 2.7 / 4.6 at depths 3 / 5 / 7), while the LLM router stays nearly flat (0.25 / 0.75 / 0.83) and Oracle fully contains (0 / 0 / 0). At depth 7 the LLM router cuts cascade radius ∼**5.5**× versus the baselines. This suggests that containment benefits can become larger in deeper pipelines, while the deployable-routing version remains future work (Figure 4). The policy CSVs are committed under `data/measured/`.

Ablation: the model, or the hint? The rows above hand the router the trusted upstream value — a possible information leak. Re-running Exp 2 as an independent, larger sweep (N=180; its LLM estimate 0.67 is a separate sample from Table 7's N=75 value 0.83) with an added `llm_noupstream` policy (same self-correction, *no* trusted-upstream hint) collapses latent recovery from **0.67 [0.46, 0.83]** to **0.08 [0.00, 0.21]**, down to the baseline level (paired drop **0.58 [0.33, 0.79],** p=5.19×10^{-4}, n=24 pairs). The LLM policy's containment is thus *mostly the trusted-upstream signal, not autonomous detection*: we read the LLM column as a trusted-state probe, not evidence that an LLM router autonomously contains latent cascades (`exp2_ablation_real.csv`).

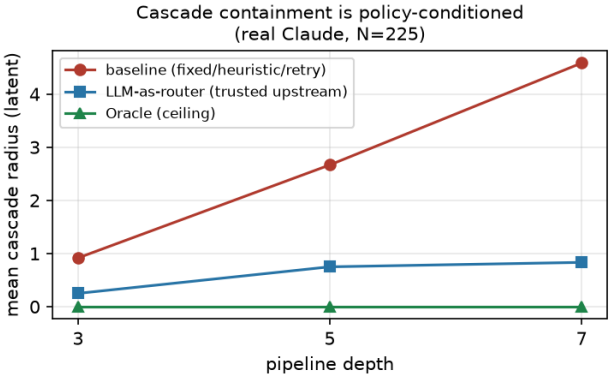


Figure 4: Experiment 3 — mean latent cascade radius vs. pipeline depth per routing policy; baseline grows with depth while the trusted-state LLM probe stays nearly flat and Oracle fully contains (real Claude, N=225).

6 DISCUSSION

Experiment 1 establishes the paper's first reproducible claim: routing reliability depends on *mechanism*. Experiments 2 and 3 then deliver the central behavioral finding: the agent's failure handling *splits sharply by fault type*. A tool-invocation fault is fully recovered (recovery 1.0) — the agent recomputes the value by hand — ambiguous delegation is *partially* recovered (0.30, the agent sometimes infers the intended operation), and the remaining three latent/semantic modes are *never* recovered (0.0 final-task success); and **retry does not repair the latent modes**: it reproduces the failure and only lengthens time-to-detection, so *detection/attribution*, not blind retry, is the necessary containment mechanism. Crucially this split is *model behavior, not a construct artifact*: it survives reframing the identical computation as a loan-approval workflow — the failure-mode ordering persists while absolute rates shift with context (Fig. 2). On top of this we quantify a structural signature prior reliability benchmarks leave unmeasured: cascade radius grows with pipeline depth (0.9 to 4.7 across depths 3–7, near-linear); on this verifiable chain that growth is partly by construction (§7), so we read it as corroborating the depth-scaling of latent cascades rather than as the headline. The policy-conditioned extension shows the potential value of trusted-state repair, but its deployable interpretation is deliberately limited (§7). These are mechanism probes on a controlled chain; domain-workflow validation is the next step.

7 LIMITATIONS

- The Exp 1 diagnostic is small (26 cases) and *saturates* for a capable model — it is a mechanism probe, not a difficulty benchmark; the larger workflow-suite results are needed for a graded comparison. Its labels are author-provided; a two-annotator protocol with reported κ is future work for the larger suite.
- Workflows are synthetically generated; real-world pipeline validation is future work.
- **Single model reported** (Claude Sonnet 4.6); every measured number is capped by this. The harness is model-agnostic — model id is an environment variable and `scripts/run_exp2_crossmode` re-runs the five-mode probe on any set of models into a pooled comparison — so the expectation the design targets is that the three-tier *ordering* (tool recovered > ambiguous partial > latent never) is model-invariant while absolute rates move, mirroring the domain-reframing result; a full cross-model sweep is future work.
- **Modest samples.** Per-mode aggregates are n=30 (Exp 2) and n=120/depth (Exp 3); the fully-deterministic cells still yield zero-width CIs by construction (a corrupted latent state always fails the final task), while the stochastic cells now carry informative intervals. Larger, more diverse workflow suites remain future work.
- **Construct validity of Exp 2/3.** The failure-recovery and cascade experiments run on a *verifiable arithmetic dependency chain*, chosen for clean ground truth, not a domain workflow. In this construct cascade radius is **partly by design** — a downstream stage consumes the upstream value, so a latent corruption propagates to ~(depth − 2) stages — so we present Exp 2/3 as **controlled mechanism probes**. The evidence that this measures model behavior rather than a tautology is the non-determinism we observe (ambiguous delegation stays below the structural maximum at every depth — mean cascade 2.2 vs. 3 at depth 5 — and recovers 0.30 of the time) and the domain-grounded re-run (Figure 2): the failure-mode *ordering* mostly persists across framings while absolute rates shift with context. Full finance / HR / DevOps suite validation remains future work.
- **Single LLM, not a literal multi-agent system.** Exp 2/3 execute one Claude agent over a staged chain with prompt-level fault injection — simulating orchestration failure modes rather than wiring N independent agents; a real multi-agent harness is future work.
- **LLM-policy semantics are a modelling assumption.** The policy-conditioned results define the LLM router as a stronger pass given the trusted upstream value and licensed to self-correct; the absolute LLM numbers depend on this choice. The Oracle (gold-repair ceiling) and baseline columns are unambiguous, but the LLM column should be read as a trusted-state self-correction probe, not yet as a deployable routing estimate. To isolate whether the LLM gain reflects the model detecting the fault versus being handed the trusted upstream, we ran an ablation policy (`llm_noupstream`) that drops the trusted-upstream hint: latent recovery collapses to near-baseline (§5.5), confirming the LLM column is a trusted-state probe, not autonomous detection.

8 ETHICS AND BROADER IMPACT

OrchestraBench evaluates the *reliability* of automated orchestration. Improving failure attribution and cascade containment is intended to make production AI systems **safer, more auditable, and cheaper to operate** — surfacing silent failures before deployment rather than after.

- *Leaderboard overfitting*: a public reliability benchmark can incentivize tuning to the injected failure modes. Mitigation: seed-controlled, **held-out** failure scenarios, and reporting cost alongside accuracy where deployment-cost comparisons are made.
- *Dual use*: cataloguing how orchestrators fail could inform adversarial prompting. Mitigation: the injected modes are already public (MAST); we add measurement, not new attack surface.
- *Over-trust*: a high score must not be read as production safety. We state scope limits (synthetic workflows; mechanism probes) explicitly.

AI-use disclosure. Generative AI tools were used for limited editorial and coding assistance; all results, citations, and claims were checked against committed code, data, and cited records.

No human-subjects data is used; all workflows are synthetic and include no real applicants, employees, customers, or proprietary records.

9 REPRODUCIBILITY

Failure scenarios are regenerable from fixed seeds, and every reported aggregate is recomputable from committed artifacts. The Exp 1 reproduction script regenerates the §5.1 table from the committed gold set (offline baselines need no key; the LLM row runs when the Anthropic API key is set), and `scripts/run_exp1_baselines.py` recomputes the Fixed / Heuristic / TF-IDF rows and Fig. 1 fully offline. A model-agnostic cross-model harness (`scripts/run_exp2_crossmodel.py`) re-runs the five-mode probe on any set of models into a pooled comparison table. The measured analysis script recomputes every Exp 2/3/4 and policy-probe number above (per-mode recovery, cascade radius by depth, decomposition fidelity, and trusted-state ablation) with bootstrap 95% CIs and exact paired permutation tests, directly from the committed CSVs (offline, no key). Code is Apache-2.0; data and failure scenarios are seeded; the full test suite (151 tests) runs offline on Python 3.10/3.11/3.12. Total real-LLM cost across the project is ≈ $4 (Sonnet 4.6; short arithmetic prompts), so reproduction is negligible.

10 CONCLUSION

OrchestraBench reframes multi-agent evaluation from "did the task succeed?" to "did the orchestrator route, recover, and decompose reliably?" Experiment 1 delivers a clean, reproducible result — model-driven routing closes a 0%→100% adversarial gap that keyword routing cannot — while the failure-injection experiments surface, on a controlled chain, the reliability axis production teams most need: a sharp three-tier fault split (tool faults fully recovered, ambiguous delegation partially recovered, three semantic faults never recovered and beyond the reach of retry) that persists under domain reframing, with cascade radius growing with pipeline depth (0.9 to 4.7 across depths 3–7) as a corroborating structural signature.

REFERENCES

[1] Anonymous (concurrent work). 2026. IntentBench: Intent Specification as a First-Class Evaluation Object. Concurrent spec-layer benchmark; complementary, cross-cite.

[2] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657 [cs.AI] Introduces the MAST taxonomy: 14 failure modes in 3 categories; MAST-Data 1,600+ traces over 7 frameworks; Cohen's kappa = 0.88.

[3] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. 2026. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems. arXiv:2604.22708 220 annotated failure traces; full-execution-observability attribution.

[4] Jin Jia, Zhiling Deng, Zhuangbin Chen, Yingqi Wang, and Zibin Zheng. 2026. MAS-FIRE: Fault Injection and Reliability Evaluation for LLM-Based Multi-Agent Systems. arXiv:2602.19843 15 fault types (8 intra-/7 inter-agent); injection via prompt / response / message-routing; tested on MetaGPT, Table-Critic, CAMEL.

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.06770; 2,294 real GitHub issues over 12 Python repos.

[6] Siddhant Kulkarni and Yukta Kulkarni. 2026. Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies. arXiv:2603.22651 10k SEC filings; reflexive F1 0.943 @2.3x, hierarchical 0.921 @1.4x.

[7] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations (ICLR)*. arXiv:2308.03688; 8 environments, 27 LLMs.

[8] Aaron Xuxiang Tian, Ruofan Zhang, Jiayao Tang, Young Min Cho, Xueqian Li, Qiang Yi, Ji Wang, Zhunping Zhang, Danrui Qi, Zekun Li, Xingyu Xiang, Sharath Chandra Guntuku, Lyle Ungar, Tianyu Shi, and Chi Wang. 2025. Beyond the Strongest LLM: Multi-Turn Multi-Agent Orchestration vs. Single LLMs on Benchmarks. arXiv:2509.23537 GPQA-Diamond: at least one agent correct in 95.5% of cases vs. 87.4% orchestrated.

[9] Yixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. 2025. OdysseyBench: Evaluating LLM Agents on Long-Horizon Complex Office Application Workflows. arXiv:2508.09124 602 long-horizon office tasks (300 real + 302 synthesized).

[10] Yizhe Xie, Congcong Zhu, Xinyue Zhang, Tianqing Zhu, Dayong Ye, Minfeng Qi, Huajie Chen, and Wanlei Zhou. 2026. From Spark to Fire: Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Collaboration. arXiv:2603.04474 Error cascades in agent pipelines.

[11] Geunbin Yu. 2026. AdaptOrch: Task-Adaptive Multi-Agent Orchestration in the Era of LLM Performance Convergence. arXiv:2602.16873 Task-adaptive orchestration method (+12–23%).

[12] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning (ICML), Spotlight.* Who&When dataset: failure logs from 127 multi-agent systems. arXiv:2505.00212.